The benchmark: what is actually being measured

`pyperformance/bm_deepcopy` registers **three** independent timed workloads:

Sub-benchmark	Payload	What it stresses
`deepcopy`	small dict + a dataclass `A`	the everyday case
`deepcopy_reduce`	object defining `__reduce__`	the pickle-protocol path
`deepcopy_memo`	`{'a':(L,L,L), 'b':[L]*100}`	the memo (shared refs)

Architecture of the module

- Each workload is a plain function `benchmark*(n)`.
- `pyperf.perf_counter()` brackets *only* the `copy.deepcopy` calls — loop and setup cost are excluded.
- Registered via `runner.bench_time_func('<name>', fn)`; `pyperf` calibrates the loop count, then runs 20 processes × 3 values.
- In `deepcopy_memo` one 100-element list appears **103 times** — a correct copy must yield *one* new list shared by all 103 slots.



How the built-in `copy.deepcopy` works

For every object it visits, `copy.py` does:

1. **memo lookup** — `memo[id(x)]`; if present, return the existing copy. This is what preserves shared references and terminates on cycles.
2. **dispatch** — `_deepcopy_dispatch[type(x)]`, an *exact-type* dict: atomics return `x` itself, list/tuple/dict get dedicated copiers.
3. **fallback** — `__deepcopy__` hook, else `__reduce_ex__(4)` → `_reconstruct()`.
4. **memoize** + `_keep_alive`.

The cost: `copy.py` is *pure Python* and recurses **once per element**. Copying `[1]*100` costs 100 Python calls that each allocate a frame, hash a type and probe the memo — only to return the integer unchanged.

```
def deepcopy(x, memo=None, _nil=[]):
    if memo is None: memo = {}
    d = id(x)
    y = memo.get(d, _nil)
    if y is not _nil:
        return y # shared ref
    cls = type(x)
    copier = _deepcopy_dispatch.get(cls)
    if copier is not None:
        y = copier(x, memo)
    else: ... # or __reduce_ex__
    if y is not x:
        memo[d] = y
        _keep_alive(x, memo)
    return y
```



Where the time goes: profile and counters

[IMAGE PLACEHOLDER]

Baseline flame graph
(pyspy_flamegraph_baseline.svg)

Widest Python frames, 7,188 samples:
`deepcopy` 238 | `_deepcopy_dict` 123 | `_deepcopy_list` 66 | `_reconstruct` 64 | `_deepcopy_tuple` 39 — all of it inside `copy.py`.

`perf` **self-time, baseline**

Self	Symbol
20.95%	`_PyEval_EvalFrameDefault`
5.19%	`call_function`
2.66%	`_PyEval_MakeFrameVector`
2.30%	`_PyObject_VectorcallTstate`
2.07%	`lookdict`
1.87%	`frame_dealloc`

Reading: no hashing, no big `memcpy`, no cache thrashing — it is *frame creation, frame teardown and bytecode dispatch*.

IPC = 2.28 confirms it: the baseline is not stalling on memory, it is simply **executing too many instructions** (**196,381** instructions per `deepcopy`).

Note: `-call-graph fp` yields *no* usable stacks on an `-Og` debug build; `dwarf` was required.

`fastdeepcopy`: a C walker for built-in containers

Re-implements the traversal in C for `dict`, `list`, `tuple` and atomic immutables. The win is *not* "C is fast" — it is that recursion no longer costs **a Python stack frame per element**.

Correctness obligations (refcounting / GC):

- **Shared memo** with `copy.deepcopy`, so the fallback path stays consistent.
- `_keep_alive` **mirrored** — without it a temporary original can be collected mid-copy and its `id()` *reused*, silently aliasing two unrelated copies.
- **Strict refcount discipline**: `PyTuple_SET_ITEM` steals, `PyList_GET_ITEM` borrows.
- `Py_EnterRecursiveCall` guards the C stack.
- New lists are **pre-filled with** `None` — never `NULL` slots reachable through the memo.

```c
/* Mirror of copy._keep_alive(): pin the
   original so its id() cannot be recycled
   while the copy is still in progress. */
static int
keep_alive(PyObject *x, PyObject *memo)
{
    PyObject *k = PyLong_FromVoidPtr(memo);
    PyObject *lst = /* borrowed */
        PyDict_GetItemWithError(memo, k);
    if (lst != NULL) {
        int rc = PyList_Append(lst, x);
        Py_DECREF(k);
        return rc;
    }
    ...
}
```

Checked on a `Py_DEBUG` build: `sys.gettotalrefcount()` drifts by **2** over 500 copies of a nested structure — no leak.

Fast path, not a replacement: delegation to `copy.py`

Three gates, in order:

1. **Exact type only.** `Py_IS_TYPE(x, &PyDict_Type)`, never `PyDict_Check` — a `dict` subclass may carry state only `__reduce_ex__` knows about, so it must take the generic route.
2. **`__deepcopy__` hook** is honoured exactly as `copy.py` does, so class-level optimizations still work through the C path.
3. **Everything else is delegated** to `copy.deepcopy(x, memo)` — *with the same memo object*, so identity and cycle handling stay globally consistent.

Tuples reproduce `_deepcopy_tuple`'s subtle rule: if no element actually changed, **return the original tuple**.

Net effect: unknown input is never handled *differently*, only known input is handled *faster*.

```
if (t == &PyList_Type)
    return deepcopy_list(x, memo);
if (t == &PyDict_Type)
    return deepcopy_dict(x, memo);
if (t == &PyTuple_Type)
    return deepcopy_tuple(x, memo);
/* user hook, like copy.deepcopy does */
hook = PyObject_GetAttr(x,
     str_deepcopy_hook);
if (hook != NULL) { ... }
/* anything else: hand it to the stdlib,
   sharing our memo */
return PyObject_CallFunctionObjArgs(
          copy_deepcopy, x, memo, NULL);
```

[IMAGE PLACEHOLDER]

Fast path vs. delegation

Optimization at the Python class level

A class that knows its own layout can copy itself far more cheaply than the generic `__reduce_ex__` / `_reconstruct` machinery.

Fast but WRONG

```
def __deepcopy__(self, memo):
    new_lst = self.lst.copy() # shallow!
    new_obj = A(self.string,
                new_lst,
                self.boolean)
    memo[id(self)] = new_obj
    return new_obj
```

- nested mutables stay **shared** with the original
- a subclass is silently downgraded to `A`
- self-referential objects break

Fast and CORRECT

```
def __deepcopy__(self, memo):
    cls = self.__class__
    new = cls.__new__(cls) # keeps subclass
    memo[id(self)] = new # BEFORE recursing
    new.string = self.string # share
    new.boolean = self.boolean # share
    new.lst = deepcopy(self.lst, memo)
    return new
```

Every line is load-bearing: `__new__` preserves subclasses, early memo registration makes cycles terminate, and the recursive call is what makes it an actual *deep* copy.

Applied to the dataclass `A` and to the `__reduce__` class in `deepcopy_reduce`. Guarded by **47 assertions** that run the same suite against the *real* classes imported from the benchmark — putting the shallow `.copy()` back fails four of them.

Performance evaluation

Execution time (`pyperf`, 20 proc. × 3 values)

	Base	Opt.	Gain
`deepcopy`	2.00 ms	372 µs	**5.39×**
`deepcopy_reduce`	18.2 µs	4.73 µs	**3.85×**
`deepcopy_memo`	233 µs	45.4 µs	**5.13×**
Geo. mean			**4.74×**

Requirement was **7%**; the weakest row is **74%**.

[IMAGE PLACEHOLDER]

Optimized flame graph — `copy.py` is gone

Hardware counters — identical **fixed** work (340,000 copies, `perf stat -r 3`)

	Base	Opt.	Δ
Wall time	12.71 s	2.78 s	**−78.2%**
Cycles	29.41 G	6.40 G	**−78.2%**
Instructions	66.77 G	15.03 G	**−77.5%**
Branches	13.41 G	3.30 G	**−75.4%**
Branch misses	54.90 M	6.65 M	**−87.9%**
Cache refs	23.45 M	3.51 M	**−85.0%**
IPC	2.28	2.35	+3%

Instructions per copy: 196,381 → 44,195.

Measurement trap: `perf stat` over the whole `pyperf` run is *invalid*: `pyperf` calibrates the loop count to a *time budget*, so 5× faster code performs ∼5× more copies and appears to use **more** instructions (316 G vs 235 G). Hence the fixed-work driver.

IPC barely moving is the point: we did not make the machine run *better*, we gave it *less to do*.